Introdução à Computação
árvores binárias de busca

Alexandre Rademaker

árvores I

Uma árvore é um tipo de dados que representa um conjunto de nós conectados em uma estrutura hierárquica.

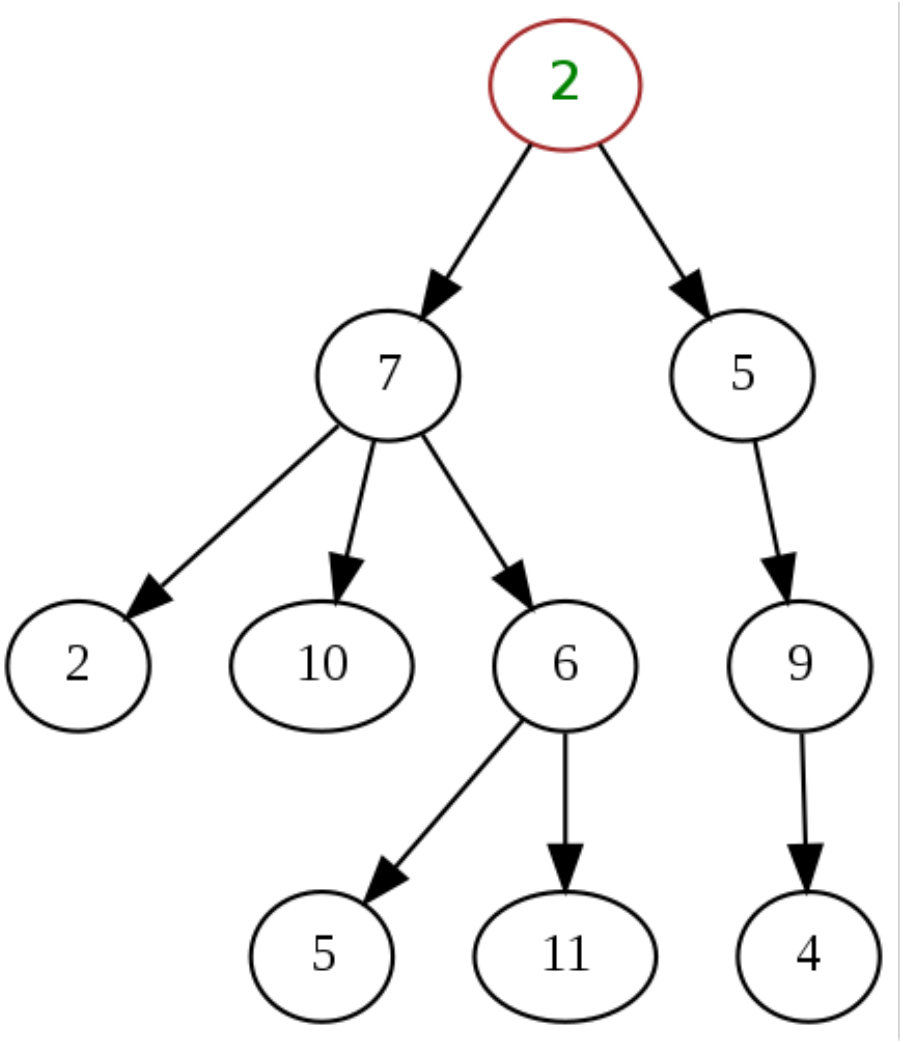
Cada nó de uma árvore pode ser conectado a um número qualquer de filhos - dependendo do tipo de árvore - mas apenas à um pai. Um único nó, chamado raiz, não tem pai.

Isto siginifica que árvores não podem conter ciclos e que cada nó pode ser considerado raiz de uma sub-árvore.

Com isso, recursão é uma técnica útil para travesia da árvore.

árvores II

- raiz
- folhas
- sub-árvores
- grau
- nível
- altura



árvores III

Árvores binárias são um tipo de árvore onde cada nó pode conter até dois filhos.

Quando a ordem dos filhos é relevante, chamamos de árvore ordenada.

Valores podem ser associados a cada nó na árvore ou, as vezes, apenas aos nós folhas - nós que não tem filhos.

árvores IV

aplicações:

- sistema de arquivos dos sistemas operacionais
- DOM de páginas HTML e arquivos XML
- árvores de busca para armazenamento de dados de forma a permitir buscas eficientes.
- árvores sintáticas no processamento de linguagem natural.

árvores V

Um tipo abstrato de dados pode ser representado de diferentes formas.

Na memória, tipicamente nós são alocados dinamicamente com ponteiros para os filhos, pais, ou ambos, além dos dados associados ao nó.

Se o número de filhos é variado, a cada nó podemos usar um array ou lista encadeada para armazenar ponteiros para os filhos.

Também podemos representar a árvore com duas listas, uma para os nós e outra para as relações pai-filho (lista de adjacencias), ou um array onde os relacionamentos (pai-filho) são determinados pelas posições no array.

árvores VI

```
'(2 (7 (2) (10) (6 (5) (11)))
    (5 (9 (4))))
```

Se nossa implementação de lista encadeada permitisse que elementos da lista fossem de qualquer tipo, incluíndo listas. . .

também poderíamos implementar uma árvore como uma lista de listas, onde o primeiro elemento de cada sublista é a raiz da sub-árvore.

Para saber mais!

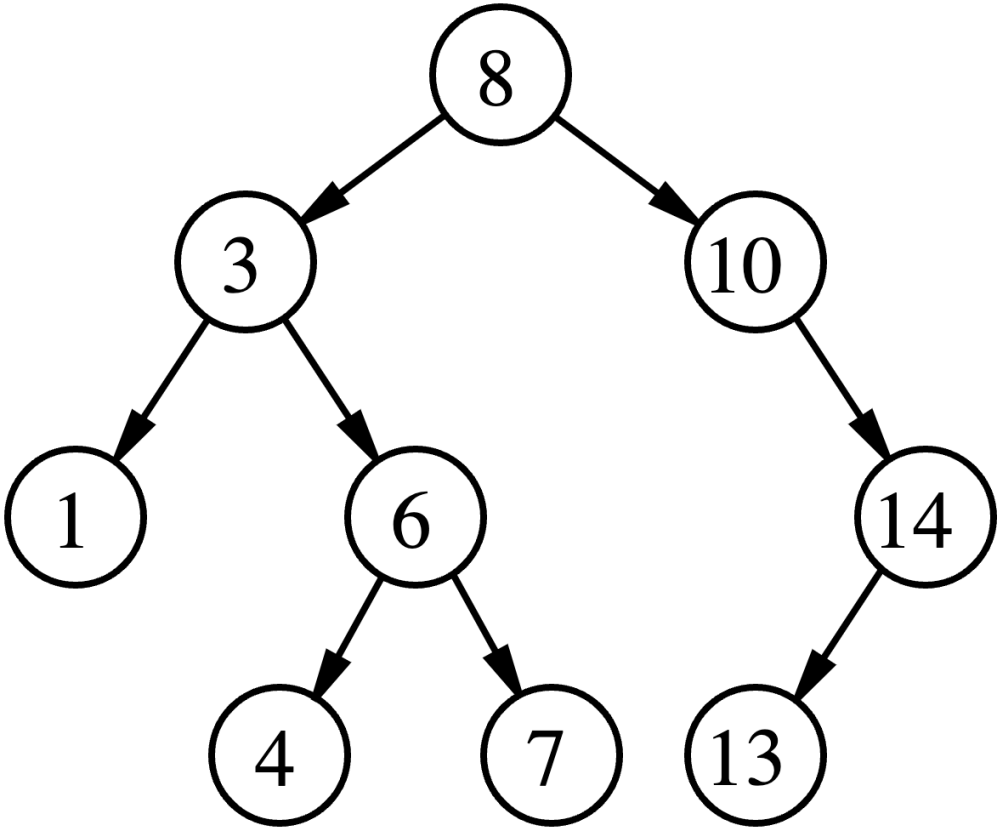
árvores binárias de busca I

Uma árvore binária de busca, também chamada de árvore binária ordenada, é uma árvore binária onde o valor de cada nó interno é maior que todos os valores na subárvore esquerda do respectivo nó e menor que as de sua subárvore direita.

A complexidade de tempo das operações na árvore de busca binária é diretamente proporcional à altura da árvore.

Buscas binárias podem ser facilmente implementadas em árvores binárias de busca.

árvores binárias de busca II



árvores binárias de busca III

Em C, os nós precisam ser agora declarados com

src/tree.c

```
1 typedef struct node {
2   int key;
3   struct node *left, *right;
4 } t_node;
```

árvores binárias de busca IV

Para inserir um novo elemento, uma função auxiliar que cria um novo nó:

src/tree.c

```
t_node* new_node(int item) {
  t_node* t = malloc(sizeof(struct node));
  assert(t != NULL);
  t->key = item;
  t->left = t->right = NULL;
  return t;
}
```

Notação `a = b = expressão`!

árvores binárias de busca V

Agora a inserção, seja o nó corrente com valor V, devemos comparar com o valor X sendo inserido. Se X menor, movemos para a árvore à esquerda. Se X maior, para árvore da direita.

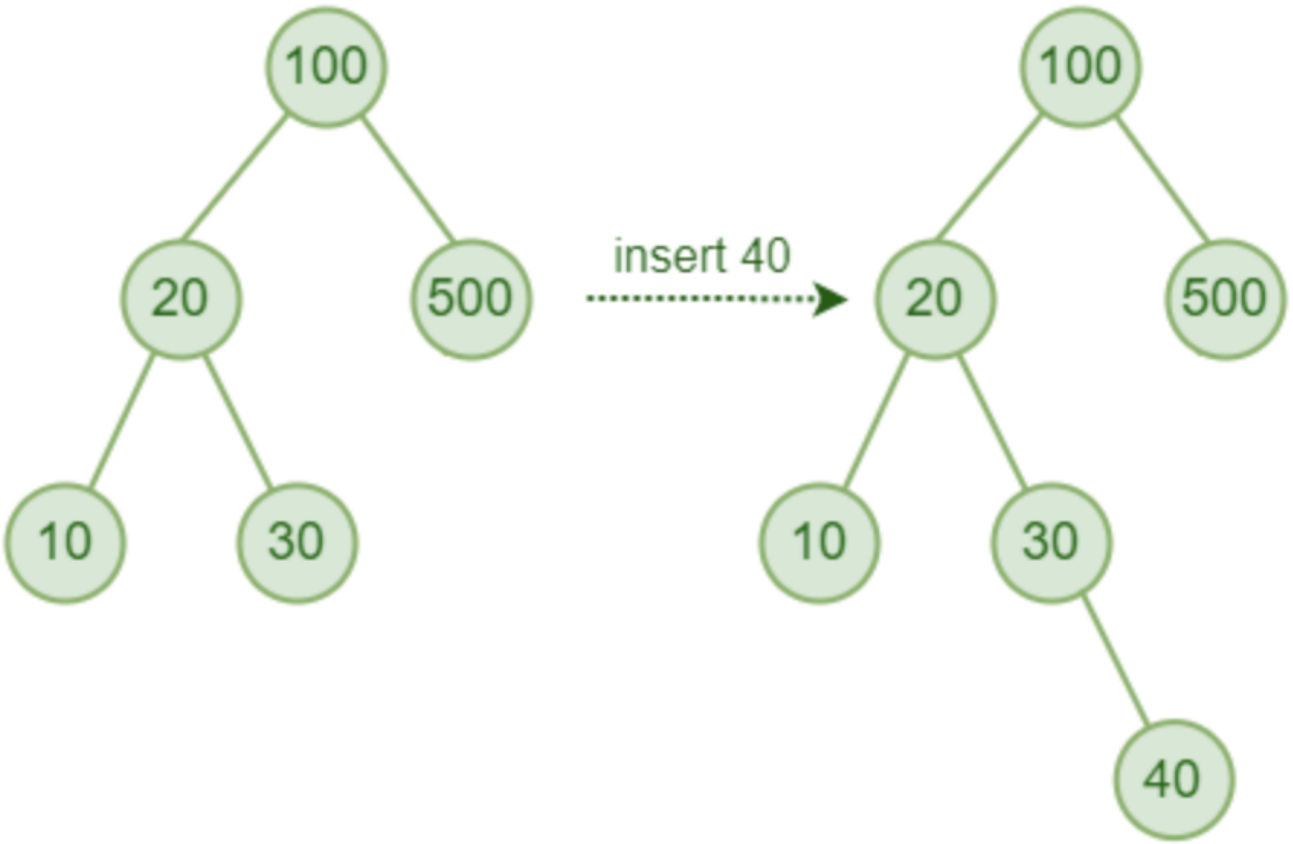
src/tree.c

```
t_node* insert(t_node* node, int key) {
  if (node == NULL)
    return new_node(key);

  if (key < node->key)
    node->left = insert(node->left, key);
  else if (key > node->key)
    node->right = insert(node->right, key);

  return node;
}
```

árvores binárias de busca VI



árvores binárias de busca VII

Nós criados de forma dinâmica precisam ser liberados.

src/tree.c

```
1 void free_tree(node *root) {
2   if (root == NULL)
3     return;
4
5   free_tree(root->left);
6   free_tree(root->right);
7   free(root);
8 }
```

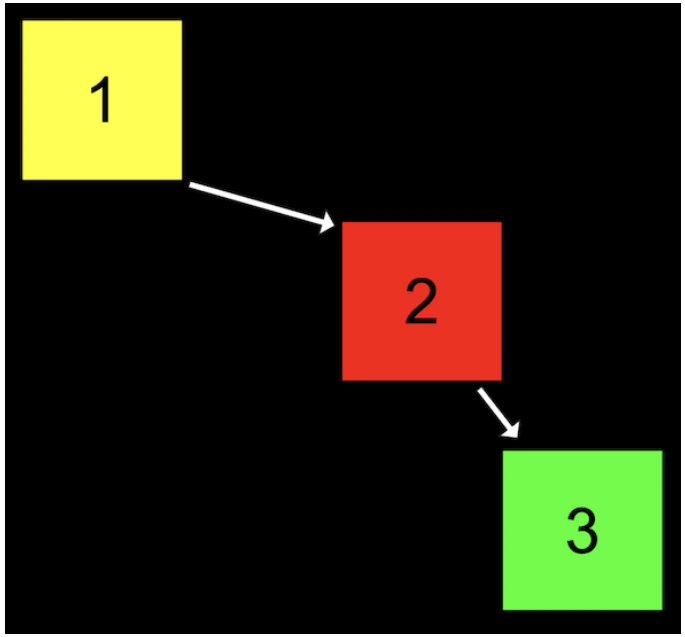
busca binária I

Usando esta estrutura, podemos então realizar buscas

src/tree.c

```
bool binary_search(t_node *root, int n) {
  if (root == NULL)
    return false;
  else if (n < root->key)
    return binary_search(root->left, n);
  else if (n > root->key)
    return binary_search(root->right, n);
  return (n == root->key);
}
```

busca binária II

Mas temos que 'balancear' a árvore, para não terminar com uma estruta ineficiente.

Em uma árvore balanceada, busca e inserção são $O(\log n)$, mas em uma árvore não balanceada, pode ser $O(n)$.